



DELEGATES

QUE ES UN DELEGATE ?

Un delegado es un tipo que representa referencias a métodos con una lista de parámetros particular y un tipo de retorno. Cuando crea una instancia de un delegado, puede asociar su instancia con cualquier método con una firma compatible y un tipo de valor devuelto. Puede invocar (o llamar) al método a través de la instancia delegada. Los delegados se utilizan para pasar métodos como argumentos a otros métodos. Los controladores de eventos no son más que métodos que se invocan a través de delegados. Usted crea un método personalizado y una clase como un control de Windows puede llamar a su método cuando ocurre un evento determinado.

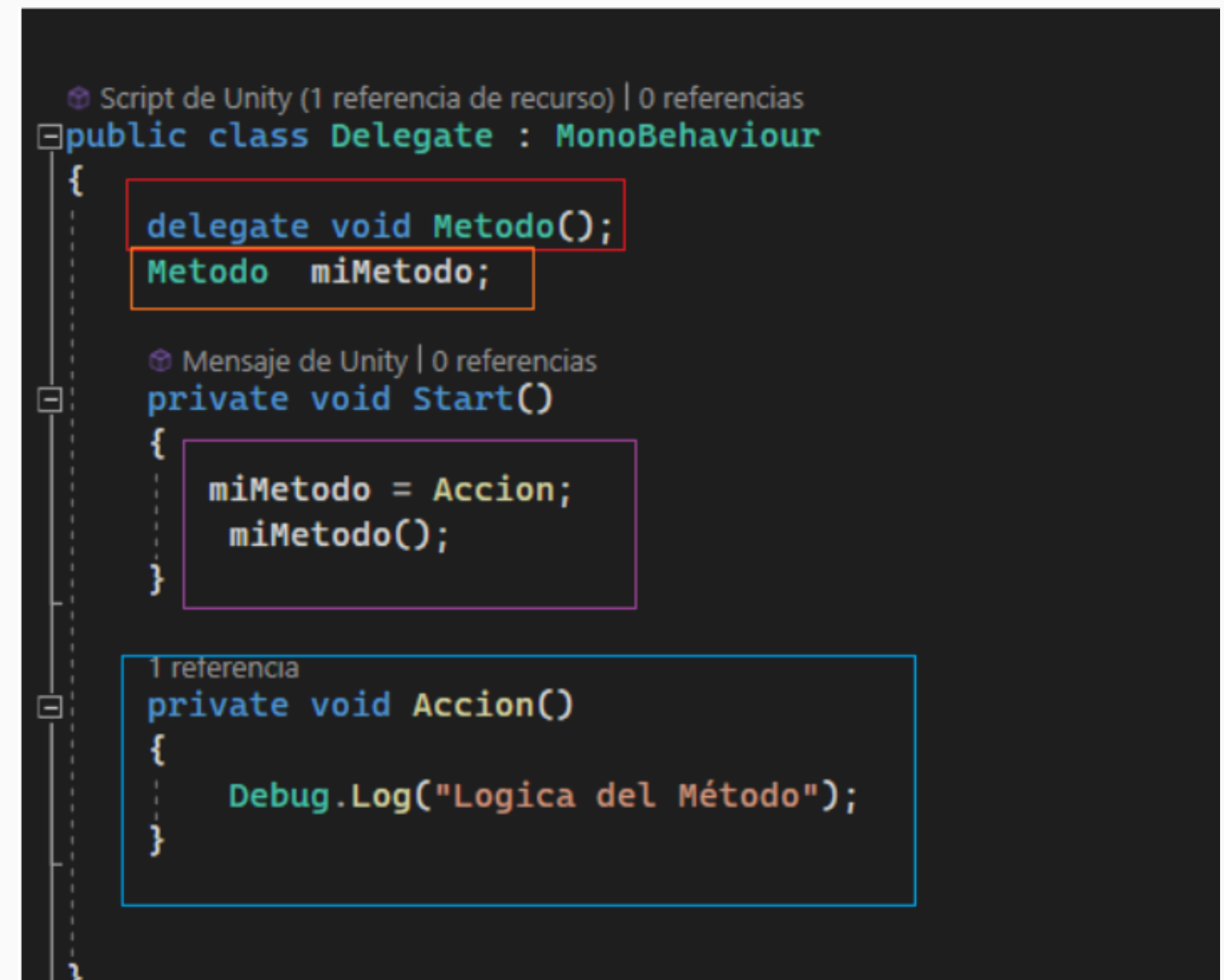
COMO SE DECLARA UN DELEGATE?

Declaración inicial: es importante utilizar la palabra clave delegate + tipo de retorno del método + nombre del método.

Declaración de la variable: esto nos va a servir para poder asignarle el método al cual vamos a llamar más adelante. Básicamente es como haríamos una declaración de una variable común.

Asignación de Método: le asignamos un método a nuestra variable recién creada, para luego invocar este método más adelante utilizando “miMetodo()”.

Método: Todo bloque de código que se encuentre dentro de este método se va ejecutar cuando llamemos a nuestra variable mencionada anteriormente.



```
Script de Unity (1 referencia de recurso) | 0 referencias
public class Delegate : MonoBehaviour
{
    delegate void Metodo();
    Metodo miMetodo;

    Mensaje de Unity | 0 referencias
    private void Start()
    {
        miMetodo = Accion;
        miMetodo();
    }

    1 referencia
    private void Accion()
    {
        Debug.Log("Logica del Método");
    }
}
```

COMO SE DECLARA UN DELEGATE?

Es importante validar si el método existe para evitar posibles errores de completamiento más adelante. Para eso lo podemos resolver de la siguiente forma:

```
⊞ Mensaje de Unity | 0 referencias  
private void Start()  
{  
    miMetodo = Accion;  
    if(miMetodo != null)  
        miMetodo();  
}
```

O si bien podemos utilizar las expresiones Lambda para reducir líneas de código.

```
⊞ Mensaje de Unity | 0 referencias  
private void Start()  
{  
    miMetodo = Accion;  
    miMetodo?.Invoke();  
}
```


COMO SE DECLARA UN DELEGATE?

Declaración inicial: en este caso utilizamos la palabra clave **delegate** + **tipo de retorno entero (int)** + **nombre del método**.

Declaración de la variable: lo llamamos **suma**, ya que es la operación que vamos a realizar más adelante.

Asignación de Método: le asignamos un método **Suma** a nuestra variable recién creada Y luego utilizamos un “**Debug.Log**” para llamarlo y realizar diferentes cálculos.

Método: Logica de calculo de suma.Es importante que el método reciba la misma cantidad y tipo de parámetros que nuestro delegate.

```
Script de Unity (1 referencia de recurso) | 0 referencias
public class Calculadora : MonoBehaviour
{
    delegate int Calcular(int a, int b);

    Calculadora suma;

    Mensaje de Unity | 0 referencias
    private void Start()
    {
        suma = Suma;
        Debug.Log("La suma es:" + suma(2,2));
        Debug.Log("La suma es:" + suma?.Invoke(4,4));
    }

    1 referencia
    int Suma(int a, int b)
    {
        return a + b;
    }
}
```

[21:37:35] La suma es:4
UnityEngine.Debug:Log (object)

[21:37:35] La suma es:8
UnityEngine.Debug:Log (object)

QUE SON LOS UNITY EVENTS ?

Los UnityEvents son maneras de permitir a los callbacks hechos por el usuario a ser persistidos desde un tiempo de edición a un tiempo de ejecución sin la necesidad de programación adicional y configuración de script.

Los UnityEvents pueden ser agregados a cualquier MonoBehaviour y son ejecutados desde código como un .net delegado estándar. Cuando UnityEvent es agregado a un MonoBehaviour éste aparece en el Inspector y callbacks persistentes pueden ser agregados.

UnityEvents tiene limitaciones similares a los delegados estándar. Es decir, contienen referencias al elemento que es el objetivo y esto evita que el objetivo sea recolectado como basura.

UNITY EVENTS : EJEMPLO

Supongamos que en nuestro juego tenemos a un personaje que tiene que recolectar todas las monedas que se encuentran dentro del juego y las tiene que mostrar en pantalla.

En este código el personaje toma como referencia el texto que se encuentra en pantalla y lo actualiza cada vez que colisiona contra una moneda.



UNITY EVENTS : EJEMPLO

En este código el personaje toma como referencia el texto que se encuentra en pantalla y lo actualiza cada vez que colisiona contra una moneda.

```
Script de Unity (1 referencia de recurso) | 0 referencias
public class PlayerMov : MonoBehaviour
{
    private Rigidbody2D rb2d;
    public float velocidad = 5;
    public Text MonedaText;
    int MonedaCont;

    Mensaje de Unity | 0 referencias
    void Start()
    {
        rb2d = GetComponent<Rigidbody2D>();
    }

    Mensaje de Unity | 0 referencias
    void Update()
    {
        MonedaText.text = "Monedas:" + MonedaCont.ToString();
        Caminar();
    }

    1 referencia
    void Caminar()...

    Mensaje de Unity | 0 referencias
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.CompareTag("Moneda"))
        {
            MonedaCont++;
        }
    }
}
```


UNITY EVENTS : EJEMPLO

Primero creamos el script de la moneda, donde inicializamos un delegate llamado MonedaEvent y cada vez que colisiona contra el jugador es invocado.

¿Qué quiere decir con esto? que todos los script que estén suscritos a Moneda Event van a activar sus métodos asignados cuando el método MonedaEvent es invocado.

```
Script de Unity (3 referencias de recurso) | 2 referencias
public class Moneda : MonoBehaviour
{
    public delegate void MonedaEvent();
    public static MonedaEvent monedaEvent;

    Mensaje de Unity | 0 referencias
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.CompareTag("Player"))
        {
            monedaEvent?.Invoke();
            Destroy(this.gameObject);
        }
    }
}
```

UNITY EVENTS : EJEMPLO

Luego creamos un script para mostrar en pantalla en tiempo real.

Como verán no es necesario tener una referencia previa para acceder al script de la Moneda.

Para que nuestro Script se suscriba al evento de la moneda, es importante dos cosas :

1. Debemos tener la referencia previa del script + nombre de la variable después de un punto y utilizamos “+=” + el método que se va invocar cada vez que se active el evento.

1. La suscripción se tiene que hacer dentro de la función OnEnable() esto nos permite evitar futuros errores más adelante. Y tenemos que suscribir con la función OnDisable() ya que si se pierde la referencia del evento y seguimos suscrito a él van a generar errores.

```
Script de Unity (1 referencia de recurso) | 0 referencias
6 public class MonedaManager : MonoBehaviour
7 {
8     [SerializeField]
9     private Text Monedatext;
10    private int Contador;
11
12    2 referencias
13    void MonedaText()
14    {
15        Contador++;
16        Monedatext.text = "MONEDAS:" + Contador;
17    }
18
19    Mensaje de Unity | 0 referencias
20    private void OnEnable()
21    {
22        Moneda.monedaEvent += MonedaText;
23    }
24
25    Mensaje de Unity | 0 referencias
26    private void OnDisable()
27    {
28        Moneda.monedaEvent -= MonedaText;
29    }
30 }
```

UNITY EVENTS : EJEMPLO

Luego creamos un script para mostrar en pantalla en tiempo real.

Como verán no es necesario tener una referencia previa para acceder al script de la Moneda.

Para que nuestro Script se suscriba al evento de la moneda, es importante dos cosas :

1. Debemos tener la referencia previa del script + nombre de la variable después de un punto y utilizamos "+=" + el método que se va invocar cada vez que se active el evento.

1. La suscripción se tiene que hacer dentro de la función OnEnable() esto nos permite evitar futuros errores más adelante. Y tenemos que suscribir con la función OnDisable() ya que si se pierde la referencia del evento y seguimos suscrito a él van a generar errores.

```
Script de Unity (1 referencia de recurso) | 0 referencias
6 public class MonedaManager : MonoBehaviour
7 {
8     [SerializeField]
9     private Text Monedatext;
10    private int Contador;
11
12    2 referencias
13    void MonedaText()
14    {
15        Contador++;
16        Monedatext.text = "MONEDAS:" + Contador;
17    }
18
19    Mensaje de Unity | 0 referencias
20    private void OnEnable()
21    {
22        Moneda.monedaEvent += MonedaText;
23    }
24
25    Mensaje de Unity | 0 referencias
26    private void OnDisable()
27    {
28        Moneda.monedaEvent -= MonedaText;
29    }
30 }
```

UNITY EVENTS : EJEMPLO

```
Script de Unity (1 referencia de recurso) | 0 referencias
public class PlayerMov : MonoBehaviour
{
    private Rigidbody2D rb2d;
    public float velocidad = 5;

    Mensaje de Unity | 0 referencias
    void Start()
    {
        rb2d = GetComponent<Rigidbody2D>();
    }

    Mensaje de Unity | 0 referencias
    void Update()
    {
        Caminar();
    }

    1 referencia
    void Caminar()
    {
        var Movimiento_X = Input.GetAxisRaw("Horizontal");

        rb2d.velocity = new Vector2(Movimiento_X * velocidad, 0);
    }
}
```

Y listo! ya tenemos nuestro juego funcionando a la perfección.



Eliminamos el código sobrante dentro de nuestro jugador.

TIP: Podemos utilizar Action para reducir líneas de código.

Una acción es básicamente una interfaz para cualquier método que no tiene retorno ni parámetros de entrada, eso es todo.

Por lo tanto, puede asignar a su variable de acción un método anónimo (como en la primera respuesta), cualquier otro método expuesto en sus scripts que cumpla con los requisitos de firma de los parámetros nulos/vacíos.

```
Script de Unity (3 referencias de recurso) | 2 referencias
public class Moneda : MonoBehaviour
{
    public static event Action<int> monedaEvent;

    Mensaje de Unity | 0 referencias
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.CompareTag("Player"))
        {
            monedaEvent?.Invoke(5);
            Destroy(this.gameObject);
        }
    }
}
```


UNITY EVENTS : EJEMPLO

Es importante que el método al cual está suscrito pida un parámetro del mismo valor que envía el evento para ser invocado.

```
Script de Unity (1 referencia de recurso) | 0 referencias
public class MonedaManager : MonoBehaviour
{
    public Text Monedatext;
    private int score;

    2 referencias
    void MonedaText(int cantidad)
    {
        score += cantidad;
        Monedatext.text = "MONEDAS:" + score;
    }

    Mensaje de Unity | 0 referencias
    private void OnEnable()
    {
        Moneda.monedaEvent += MonedaText;
    }

    Mensaje de Unity | 0 referencias
    private void OnDisable()
    {
        Moneda.monedaEvent -= MonedaText;
    }
}
```



CONCLUSIÓN

